Comprehensive Guide to Multinomial Naive Bayes

Parth Bansal

June 17, 2025

---

1. Problem Setup

In text classification, we convert each document to a bag-of-words count vector. Given a corpus of n documents, we first build a vocabulary of size D from the most frequent or informative words. Each document i is then represented by a count vector $x^{(i)} \in \mathbb{Z}_{\geq 0}^{D}$, where:

$$x_j^{(i)} = \text{number of times word } j \text{ appears in document } i.$$

Stack these row vectors to form the count matrix:

$$X = \begin{pmatrix} x^{(1)T} \\ \vdots \\ x^{(n)T} \end{pmatrix} \in \mathbb{Z}_{\geq 0}^{n \times D}.$$

We also have binary labels:

$$y = (y^{(1)}, \ldots, y^{(n)})^T, \quad y^{(i)} \in \{0, 1\},$$

where $y^{(i)} = 1$ denotes the positive class (e.g. spam) and 0 the negative class (e.g. ham). This representation ignores word order but captures token frequencies.

2. Model & Parameters

Multinomial Naive Bayes models each document as follows:

1. Draw a class label $y^{(i)} \sim \text{Bernoulli}(\phi_y)$, where:

$$\phi_y = P(y = 1) = \frac{1}{n} \sum_{i=1}^{n} y^{(i)}, \quad P(y = 0) = 1 - \phi_y.$$

2. Given class y, draw $N_i = \sum_j x_j^{(i)}$ word tokens independently from a multinomial distribution over D words with parameters $\{\phi_{j|y}\}_{j=1}^{D}$, where:

$$\phi_{j|y} = P(\text{word} = j \mid y), \quad \sum_{j=1}^{D} \phi_{j|y} = 1.$$

The probability of observing counts $x^{(i)}$ given class y is:

$$P(x^{(i)} \mid y) = \frac{(N_i)!}{\prod_j x_j^{(i)}!} \prod_{j=1}^{D} \phi_{j|y}^{x_j^{(i)}},$$

but the multinomial coefficient can be omitted during classification as it does not depend on y.

3. Maximum Likelihood Estimation

Given training data (X, y), estimate parameters by maximizing the joint likelihood. Define:

$$N_1 = \sum_{i=1}^{n} y^{(i)}, \quad N_0 = n - N_1.$$

Compute class-conditional counts:

$$pos\_counts_j = \sum_{i:y^{(i)}=1} x_j^{(i)}, \quad neg\_counts_j = \sum_{i:y^{(i)}=0} x_j^{(i)}.$$

Total tokens in each class:

$$T_1 = \sum_{j=1}^{D} pos\_counts_j, \quad T_0 = \sum_{j=1}^{D} neg\_counts_j.$$

Without smoothing, MLEs are:

$$\hat{\phi}_y = \frac{N_1}{n}, \quad \hat{\phi}_{j|1} = \frac{pos\_counts_j}{T_1}, \quad \hat{\phi}_{j|0} = \frac{neg\_counts_j}{T_0}.$$

To prevent zero probabilities for rare words, apply Laplace smoothing ($\alpha > 0$):

$$\hat{\phi}_{j|1} = \frac{pos\_counts_j + \alpha}{T_1 + \alpha D}, \quad \hat{\phi}_{j|0} = \frac{neg\_counts_j + \alpha}{T_0 + \alpha D}.$$

Here α acts as pseudo-counts, ensuring every word has non-zero probability.

4. Predictive Distribution

For a new document with count vector x, the posterior over classes is:

$$P(y = 1 \mid x) = \frac{P(y = 1)\, P(x \mid y = 1)}{\sum_{y' \in \{0,1\}} P(y')\, P(x \mid y')}.$$

Working in log-space to avoid numerical underflow, define:

$$\ell_1 = \log P(y = 1) + \sum_{j=1}^{D} x_j \log \hat{\phi}_{j|1},$$

$$\ell_0 = \log P(y = 0) + \sum_{j=1}^{D} x_j \log \hat{\phi}_{j|0}.$$

Then use the log-sum-exp trick:

$$\log Z = \text{logaddexp}(\ell_1, \ell_0), \quad P(y = 1 \mid x) = \exp(\ell_1 - \log Z).$$

For hard classification, choose $y = 1$ if $\ell_1 > \ell_0$, else $y = 0$.

5. Implementation Details

Implementation steps:

1. **Tokenization and dictionary creation:** extract words, filter rare tokens.
2. **Matrix construction:** build X by counting word occurrences per document.
3. **Training:**
   - $\phi_y = \text{np.mean}(labels)$.
   - Split X into X_{pos}, X_{neg} using boolean masks.
   - Compute $pos\_counts = X_{pos}.\text{sum}(axis = 0)$ and similarly for negative.
   - Compute smoothed $\phi_{j|1}, \phi_{j|0}$.
4. **Prediction:**
   - Precompute $\log \phi_{\cdot}$ arrays.
   - For each x, compute ℓ_1, ℓ_0 via `np.dot(x, log_phi)`.
   - Normalize with `np.logaddexp` and exponentiate to get probabilities.

6. Handling Unseen Words

By design, words not in the training dictionary have no columns in X and thus count zero, contributing nothing to the log-scores. To explicitly model unseen words, one can add an "UNK" feature and count unknown tokens under it.

7. Indicative Word Ranking

To rank words by how strongly they indicate the positive class, compute the log-ratio:

$$s_j = \log \frac{\hat{\phi}_{j|1}}{\hat{\phi}_{j|0}}.$$

A large positive s_j means word j is much more prevalent in positive versus negative documents. Sort s_j descending and map top indices back to words using an inverse dictionary.